This assignment involves building a **trie** data structure which can hold a large dictionary in memory. By large, we mean over 100,000 words. For simplicity, we will assume that the words only use letters A-Z (no special characters), and will be confined to the English language (though there are many words in the list that will be provided to you which one may not recognise as English!).

The main module of the program you are being asked to write will implement a **trie**.
The operations should include

- **creating** a new (empty) trie
- **looking up** a given word in a given trie to see if a word is in the dictionary or not (a boolean value is returned.
- a function for **adding** a word to a trie (one word at a time).
- a function for reading in a large list of words from a text file, one word per line, and **adding all** the words in the list to the dictionary. (**Building** the dictionary into a trie)
- reading in a large list of words separated by commas (a comma-separated list) and/or some white space, and a**dding** them to the trie. (Alternative for **building** a trie)
- If the words given to you are in lower case, a simple routine that can turn them into upper-case for looking them up or for adding them to the trie.
- A function to **delete** a word from the trie
- A function to **delete a list of words** from the trie
- A function that given a list of words in a file (either one in a line, or comma/space separated) returns all those words that are in the trie.
- A function that given a list of words in a file (either one in a line, or comma/space separated) returns all those words *not* in the trie.

Your tasks are to implement the trie dictionary structure in Rust. You need to extensively test the find, insert and delete functions, as well as the routines to build a trie — first manually, and then using any mechanism for generating guesses (see the last two functions). This is the "command line interface" or module called "trie-cli". The module structure expected is written below.

Having built a trie, one can use it in word games. Those will be the next two (or three) assignments.

Learning outcomes expected:

- Correct choice of using *mutable* and *immutable* variables, use of **constants**; appropriate ***parameter passing*** — by reference or by copying. Please note ownership transfers.
- Familiarity with control flow — loops, conditionals, termination. Definite and indefinite iteration.
- F**unction-level abstraction**
- Choice of **appropriate data structures** — arrays / vectors, indexing. Structs and Enums. The option type.
- Development of the program as **modules**: data encapsulation.
- Use of **random** word generators for the game, and for **testing** — particularly unit testing
- documentation of the code
- **meta-documentation of the testing process.**
- **Using Cargo** appropriately to build, check, test and release the project.

```
word-trie/
├── Cargo.toml
├── trie/
│   ├── Cargo.toml
│   └── src/lib.rs
└── trie-cli/
    ├── Cargo.toml
    └── src/main.rs
```